ITA06 – MACHINE LEARNING

ASSIGNMENT 5 – CO5

NUMPY ARRAYS, PANDAS DATA ANALYSIS, DATA CLEANING, GROUPING, VISUALISATION AND REPRODUCIBLE PIPELINES

Student Name: A. Lakshmi Devi

Register Number: 192525176

Programme / Branch: AI & ML (Artificial Intelligence & Machine Learning)

Course Code: ITA0611

Course Name: Machine Learning

Assignment: 5 – CO5

Total Marks: 50

Duration: 1 Week

CONTENTS

1. Q1 – NumPy sensor data and vectorised calculations
2. Q2 – Indexing, filtering, statistics and saving
3. Q3 – Pandas DataFrame cleaning and file I/O
4. Q4 – Merge, group, filter and sort
5. Q5 – Data visualisation
6. Q6 – Reproducible analysis pipeline
7. Q7 – Diagnose, redesign and validate
8. Conclusion
9. Appendix – Reproducible Python code

INTRODUCTION

This report follows the supplied ITA06 Machine Learning Assignment 5 – CO5 brief (50 marks). It covers NumPy arrays, mathematical operations, random numbers, indexing, filtering, statistics, saving data, Pandas DataFrames, merging, grouping, sorting, plotting and reproducible analysis.

Evidence note: the assignment brief requires actual outputs from code, timing and memory measurements, plots and saved datasets. Manual calculations and executable code are provided. Machine-generated experimental outputs are not claimed as measured here; run the code and insert the resulting tables and figures before submission. The source brief (pages 1–6) specifically requires reproducible seeds, data partitions, round-trip checks and raw experimental evidence.

Q1 – REPRESENT AND COMPUTE WITH SENSOR DATA USING NUMPY ARRAYS (6 MARKS)

1.1 Create array and inspect attributes

```
import numpy as np
temps = np.array([[70,72,75,71,90],
                  [65,66,64,67,66],
                  [80,82,85,88,91],
                  [60,61,59,62,60]], dtype=np.int64)
print(temps.shape, temps.ndim, temps.size, temps.dtype, temps.nbytes)
temps32 = temps.astype(np.float32)
print(temps32.dtype, temps32.nbytes)
```

The array has shape (4,5), 2 dimensions and 20 elements. With int64, memory is 20×8=160 bytes. Converting to float32 uses 20×4=80 bytes, reducing array data memory by 50%.

1.2 Equivalent creation, Fahrenheit and day-to-day change

```
same = np.array([[70,72,75,71,90],[65,66,64,67,66],
                 [80,82,85,88,91],[60,61,59,62,60]])
print(np.array_equal(temps, same))
fahrenheit = temps * 9/5 + 32
daily_change = temps[:, 1:] - temps[:, :-1]
print(fahrenheit)
print(daily_change)
```

The Fahrenheit array remains (4,5). The day-to-day difference has shape (4,4), because each machine has four consecutive-day changes.

Machine	Day 1→2	Day 2→3	Day 3→4	Day 4→5
M1	+2	+3	−4	+19
M2	+1	−2	+3	−1
M3	+2	+3	+3	+3
M4	+1	−2	+3	−2

1.3 Broadcasting and seeded random errors

```
machine_mean = temps.mean(axis=1, keepdims=True) # shape (4,1)
centred = temps - machine_mean                   # shape (4,5)
rng = np.random.default_rng(42)
errors = rng.normal(loc=0, scale=1, size=(4,5))
noisy = temps + errors
rng2 = np.random.default_rng(42)
print(np.array_equal(errors, rng2.normal(0,1,(4,5))))
```

Broadcasting is valid because (4,5) and (4,1) are compatible: the singleton second dimension is repeated across five days. The seeded generator produces the same 4×5 error array each time the same seed and procedure are used.

1.4 NumPy mathematical functions and view versus copy

```
a = np.array([-2.0, 0.0, 1.5, 4.0])
print(np.sqrt(np.clip(a, 0, None)))
print(np.exp(a))
print(np.log(np.clip(a, 1e-6, None)))
print(np.round(a, 1))
print(np.where(a > 1, "High", "Normal"))
```

A view shares the original array's memory, so changing the view can change the source. A copy owns separate memory. Example: v=temps[:, :2] is generally a view; v[0,0]=99 can update temps. c=temps[:, :2].copy() is independent; changing c does not update temps. Use .copy() when a slice will be modified independently.

Predicted shapes: original and Fahrenheit (4,5); daily changes (4,4); machine means (4,1); centred array and random errors (4,5).

Q2 – INDEX, FILTER, SUMMARISE AND SAVE NUMERICAL DATA (6 MARKS)

2.1 Generate reproducible 50×30 data and indexing

```
rng = np.random.default_rng(2026)
data = rng.normal(loc=70, scale=5, size=(50,30))
extreme_positions = [(2,4),(10,17),(25,8),(41,22)]
for r,c in extreme_positions:
    data[r,c] += 35
last7_first10 = data[:10, -7:]          # basic slicing
selected_machines = data[[2,6,11], :]   # fancy indexing: machines 3,7,12
above_threshold = data[data > 85]       # boolean indexing
```

Basic slicing returns the last seven days for the first ten machines (10,7). Fancy indexing selects rows 3, 7 and 12 (3,30). Boolean indexing returns a one-dimensional array containing every value above the chosen threshold (85°C).

2.2 Summary statistics and axis meaning

For the full array, use np.mean, np.median, np.std, np.min, np.max and np.percentile(data,90). With axis=1, each row is reduced to one statistic per machine, giving shape (50,). With axis=0, each column is reduced to one statistic per day, giving shape (30,). Axis=1 works across columns within each machine; axis=0 works down rows for each day.

2.3 Z-score outliers and replacement

```
mu, sd = data.mean(), data.std()
z = (data - mu) / sd
mask = np.abs(z) > 3
rows, cols = np.where(mask)
clean = data.copy()
machine_medians = np.median(data, axis=1)
clean[mask] = np.broadcast_to(machine_medians[:,None], data.shape)[mask]
print(list(zip(rows,cols)), clean.shape == data.shape)
```

The inserted extreme positions are (2,4), (10,17), (25,8) and (41,22). A global z-score may also flag naturally extreme random values; compare the returned positions with the injected positions rather than assuming they are the only flagged values. Replacing each flagged cell with its own row's median preserves the (50,30) shape.

2.4 Loop versus vectorised calculation and saving

```
import time
big = np.random.default_rng(7).normal(size=(1000,1000))
t0=time.perf_counter()
loop_means=np.array([sum(row)/len(row) for row in big])
loop_time=time.perf_counter()-t0
t0=time.perf_counter()
vec_means=big.mean(axis=1)
vector_time=time.perf_counter()-t0
print(loop_time, vector_time, np.allclose(loop_means,vec_means))
np.save("clean.npy", clean)
np.savez("clean_bundle.npz", clean=clean)
np.savetxt("clean.csv", clean, delimiter=",")
a=np.load("clean.npy")
b=np.load("clean_bundle.npz")["clean"]
c=np.loadtxt("clean.csv", delimiter=",")
print(a.dtype,b.dtype,c.dtype,np.allclose(a,b),np.allclose(a,c))
```

NumPy binary formats preserve numerical dtype and are compact and fast. savetxt writes human-readable text, but it is larger, slower, and may need explicit dtype handling when loaded. For this reason, np.savetxt is a poor choice for large arrays or exact dtype preservation compared with np.save.

Mean and standard deviation are sensitive to extreme values; median is more robust and is usually least affected. Minimum and maximum are directly affected if the inserted values become the extremes. Support this conclusion with the actual summary numbers produced by the script.

Q3 – LOAD, INSPECT AND CLEAN DATA USING PANDAS DATAFRAMES (7 MARKS)

3.1 Create CSV, load and inspect

```
import pandas as pd
from io import StringIO
csv_text = """Machine_ID,Date,Line,Output_Units,Defects
M01,2026-03-01,A,480,12
M02,2026-03-01,A,455,9
M03,2026-03-01,B,N/A,15
M01,2026-03-02,A,492,
M02,2026-03-02,A,455,9
M02,2026-03-02,A,455,9
M03,2026-03-02,b,510,14
"""
```

```
df = pd.read_csv(StringIO(csv_text), na_values=["N/A"],
                 parse_dates=["Date"])
print(df.head()); df.info(); print(df.describe(include="all"))
print(df.dtypes, df.shape, df.isna().sum())
```

head() previews records; info() shows row count, non-null counts and memory; describe() summarises numerical and categorical columns; dtypes reveals incorrect data types; shape gives dimensions; isna().sum() counts missing values per column.

3.2 Cleaning steps and justification

```
before = len(df)
df = df.drop_duplicates()
duplicates_removed = before - len(df)
df["Line"] = df["Line"].str.strip().str.upper()
df["Output_Units"] = pd.to_numeric(df["Output_Units"], errors="coerce")
df["Defects"] = pd.to_numeric(df["Defects"], errors="coerce")
# Strategy A: drop rows with missing output (output is essential denominator)
drop_output = df.dropna(subset=["Output_Units"])
# Strategy B: impute missing defects with the line median
df["Defects"] = df.groupby("Line")["Defects"].transform(
    lambda s: s.fillna(s.median()))
df = drop_output.copy()
df["Defect_Rate"] = df["Defects"] / df["Output_Units"]
df = df.set_index(["Machine_ID","Date"])
```

The exact duplicate M02 record is removed (one row). Line value 'b' becomes 'B'. 'N/A' and the blank Defects cell become missing numeric values after conversion. Dropping the row with missing Output_Units is defensible because output is the denominator of Defect_Rate and cannot safely be invented. For missing Defects, a line-wise median imputation can retain the record, but it may bias variability and should be flagged. The sample contains one missing output and one missing defect value.

Selection examples: df['Output_Units'] selects one column; df.loc[('M01', pd.Timestamp('2026-03-01'))] is label-based; df.iloc[0] is position-based. A two-column boolean condition is df[(df['Line']=='A') & (df['Output_Units']>470)]. Chained indexing such as df[df.Line=='A']['Output_Units']=0 may modify a temporary object rather than the original; use .loc[condition,'Output_Units']=0.

3.3 Generate 1,000-row dataset and verify file I/O

Create 12 machine IDs, two lines, three shifts, dates across two months, integer output and defect counts from fixed-seed distributions. Deliberately set selected Output_Units and Defects values to missing and duplicate a few rows. Save to CSV, read it back with parse_dates, and compare values, row counts and dtypes after applying the same type conversions. Also demonstrate to_json/read_json or to_excel/read_excel. CSV does not store pandas dtypes exactly; dates and nullable integer types should be explicitly restored after reading.

```
rng = np.random.default_rng(123)
n=1000
syn = pd.DataFrame({
 "Machine_ID":rng.choice([f"M{i:02d}" for i in range(1,13)],n),
 "Date":pd.to_datetime("2026-01-01")+pd.to_timedelta(rng.integers(0,60,n),unit="D"),
 "Line":rng.choice(["A","B","C"],n),
 "Output_Units":rng.integers(350,600,n),
 "Defects":rng.poisson(10,n),
 "Shift":rng.choice(["Day","Evening","Night"],n)})
syn.loc[[5,55,155],"Output_Units"]=np.nan
syn.loc[[9,99],"Defects"]=np.nan
syn=pd.concat([syn,syn.iloc[[20,21]]],ignore_index=True)
syn.to_csv("production_1000.csv",index=False)
back=pd.read_csv("production_1000.csv",parse_dates=["Date"])
print(syn.shape,back.shape,back.dtypes)
```

A cleaning log should report each affected count: exact duplicate rows removed, line labels standardised, values coerced to numeric, missing values dropped or imputed. Imputing output or defects can bias later comparisons if missingness is related to machine performance. Detect this by comparing missingness rates by machine, line, shift and date, and by running a sensitivity analysis with drop versus imputation.

Q4 – COMBINE, GROUP, FILTER AND SORT DATA TO ANSWER QUESTIONS (7 MARKS)

4.1 Merges and unmatched records

```
# production is the cleaned Q3 dataset; machine_info has >=12 machines
inner = production.merge(machine_info,on="Machine_ID",how="inner",
                         indicator=True)
left = production.merge(machine_info,on="Machine_ID",how="left",
                        indicator=True)
outer = production.merge(machine_info,on="Machine_ID",how="outer",
                         indicator=True)
print(inner["_merge"].value_counts())
print(left["_merge"].value_counts())
print(outer["_merge"].value_counts())
```

Inner merge keeps only machine IDs present in both tables; left merge keeps every production row and adds machine information when available; outer merge keeps all production and machine-info IDs. The indicator column labels records as both, left_only or right_only.

Include at least one Machine_Info machine with no production and one production machine absent from Machine_Info to demonstrate the differences.

4.2 Monthly concat, groupby, pivot and rankings

```
jan = production[production["Date"].dt.month==1]
feb = production[production["Date"].dt.month==2]
combined = pd.concat([jan,feb],ignore_index=True)
summary = production.groupby("Line").agg(
 total_output=("Output_Units","sum"),
 mean_defect_rate=("Defect_Rate","mean"),
 max_defects=("Defects","max"),
 records=("Machine_ID","size"))
pivot = pd.pivot_table(production,index="Line",columns="Shift",
                       values="Defect_Rate",aggfunc="mean")
overall = production["Defect_Rate"].mean()
machine_mean = production.groupby("Machine_ID")["Defect_Rate"].mean()
above = machine_mean[machine_mean>overall]
ranking = production.groupby("Machine_ID").agg(
 mean_rate=("Defect_Rate","mean"), output=("Output_Units","sum"))
ranking = ranking.sort_values(["mean_rate","output"],
                              ascending=[False,False],kind="mergesort")
top5, bottom5 = ranking.head(5), ranking.tail(5)
```

pd.concat stacks monthly records vertically; ignore_index=True creates a fresh sequential index. Verify that the recombined row count equals the sum of monthly row counts and compare record IDs or sorted values to ensure no loss or duplication. groupby summarises categories; pivot_table lays one grouping variable across columns and is useful for a Line-by-Shift comparison. transform can attach group-level means back to every row for group filtering.

Three manager questions: (1) Which line has the highest total output? (2) Which machines have mean defect rates above the plant average? (3) Does defect rate differ by shift? Compare conclusions using left/outer merge versus dropping unmatched records. Dropping unmatched machines may exclude production data or hide missing metadata; state which rows were lost and whether the conclusion changes.

Q5 – VISUALISE DATA TO SUPPORT A DECISION (5 MARKS)

5.1 Required plots and interpretation

Plot	Question answered	Why appropriate
Line plot: daily output by Line	How does output change over dates for each line?	Shows trends and compares time series.
Sorted bar chart: mean defect rate by machine	Which machines have the highest average defect rate?	Compares discrete categories.
Histogram of Defect_Rate	What is the distribution and skew of defect rate?	Shows frequency across numeric intervals.
Box plot by Shift	Do shifts differ in median and spread of defect rate?	Shows median, quartiles and outliers.
Scatter: Output_Units vs Defects	Do defects rise with production volume?	Shows relationship and outliers; add fitted trend line.

5.2 Example plotting code

```
import matplotlib.pyplot as plt
fig,axs=plt.subplots(2,2,figsize=(12,8))
daily=production.groupby(["Date","Line"])["Output_Units"].sum().unstack()
daily.plot(ax=axs[0,0],marker="o",title="Daily Output by Line")
machine_mean.sort_values().plot(kind="bar",ax=axs[0,1],
                                title="Mean Defect Rate by Machine")
production["Defect_Rate"].plot(kind="hist",bins=15,ax=axs[1,0],
                               title="Defect Rate Distribution")
production.boxplot(column="Defect_Rate",by="Shift",ax=axs[1,1])
fig.tight_layout()
fig.savefig("production_dashboard.png",dpi=300,bbox_inches="tight")
plt.show()
```

For the scatter plot, add a least-squares trend line using np.polyfit(Output_Units, Defects, 1). A positive slope indicates defects tend to increase with output, but does not prove that higher output causes defects. Use clear titles, units, legends and date labels. A pie chart would be misleading for daily trends or continuous distributions because it hides temporal order and distribution shape.

A truncated y-axis on a bar chart can exaggerate small differences. Revise it to start at zero and show both versions with an explanation. Histograms can also mislead if bin width is chosen to exaggerate or hide patterns; state the bin count and check another reasonable choice.

Q6 – ENGINEER A REPRODUCIBLE DATA ANALYSIS PIPELINE (6 MARKS)

6.1 Functional pipeline and validation checks

```
def load_data(path):
    return pd.read_csv(path,parse_dates=["Date"])
def validate_columns(df, required):
    missing=set(required)-set(df.columns)
    if missing: raise ValueError(f"Missing columns: {sorted(missing)}")
def clean_data(df):
    validate_columns(df,["Machine_ID","Date","Line","Output_Units","Defects","Shift"])
    out=df.drop_duplicates().copy()
    out["Line"]=out["Line"].str.upper().str.strip()
    out["Output_Units"]=pd.to_numeric(out["Output_Units"],errors="coerce")
    out["Defects"]=pd.to_numeric(out["Defects"],errors="coerce")
    return out
def summarise(df):
    return df.groupby("Line").agg(
      output=("Output_Units","sum"), mean_defect=("Defect_Rate","mean"))
def run_pipeline(path):
    raw=load_data(path); cleaned=clean_data(raw)
    if cleaned["Machine_ID"].isna().any():
        raise ValueError("Machine_ID contains missing keys")
    if cleaned.duplicated(["Machine_ID","Date","Shift"]).any():
        raise ValueError("Duplicate business keys after cleaning")
    cleaned["Defect_Rate"]=cleaned["Defects"]/cleaned["Output_Units"]
    return cleaned,summarise(cleaned)
```

Add explicit merge validation such as validate='many_to_one' when production rows map to one machine-info row. Check expected row counts before and after each merge; stop if unexpected expansion occurs. Avoid hidden global variables by passing each DataFrame and configuration as function arguments.

6.2 Loop, apply and vectorised calculation

```
large=pd.DataFrame({"Output_Units":np.random.default_rng(1).integers(300,600,100000),
                    "Defects":np.random.default_rng(2).integers(0,30,100000)})
t0=time.perf_counter()
loop=[large.Defects.iloc[i]/large.Output_Units.iloc[i] for i in range(len(large))]
loop_t=time.perf_counter()-t0
t0=time.perf_counter()
applied=large.apply(lambda r:r.Defects/r.Output_Units,axis=1)
apply_t=time.perf_counter()-t0
t0=time.perf_counter()
vector=large["Defects"]/large["Output_Units"]
vector_t=time.perf_counter()-t0
print(loop_t,apply_t,vector_t,
      np.allclose(loop,applied),np.allclose(loop,vector))
```

The vectorised expression usually runs fastest because the arithmetic executes in optimised array operations without Python-level row iteration. Measure actual time on the same machine and report the values; do not invent timings.

6.3 Memory reduction and reproducibility

```
before=production.memory_usage(deep=True).sum()
production["Line"]=production["Line"].astype("category")
production["Shift"]=production["Shift"].astype("category")
production["Output_Units"]=production["Output_Units"].astype("Int32")
after=production.memory_usage(deep=True).sum()
print(before,after,(before-after)/before*100)
a1,s1=run_pipeline("production_1000.csv")
a2,s2=run_pipeline("production_1000.csv")
pd.testing.assert_frame_equal(a1,a2)
pd.testing.assert_frame_equal(s1,s2)
```

Categorical dtype is useful for repeated labels; smaller integer types reduce storage when their range is sufficient. Run the pipeline twice on the same input and compare outputs with pandas.testing.assert_frame_equal. Then change one input record and record which summary rows and values change. Checks that catch real issues include missing required columns, duplicate business keys, invalid numeric values and merge row expansion.

Q7 – DAY-4 DESIGN CHALLENGE: DIAGNOSE, REDESIGN AND VALIDATE (13 MARKS)

Selected limitation: slow row-wise computation and excessive memory use

A. Diagnose the bottleneck (2 marks)

The loop and DataFrame.apply approaches execute Python operations for each row and are slower than vectorised arithmetic on large datasets. Repeated text labels stored as general object strings also use more memory than categorical columns. This limitation is computational efficiency and memory representation.

B. Define the design objective (2 marks)

Baseline: row-wise loop/apply calculation and object-typed categorical columns. Target: vectorised Defect_Rate must produce numerically identical values, reduce calculation time by at least 50%, and reduce DataFrame memory by at least 20% after suitable dtype conversion. These are pre-registered targets to verify, not claimed measured results.

C. Design the revised analysis strategy (4 marks)

Replace row-wise operations with vectorised column expressions; convert repeated Line and Shift labels to category; use nullable integer dtypes where ranges allow; validate required columns and business keys before merging; use merge validation and explicit row-count checks; use groupby().transform for group-level filters rather than repeated loops.

```
CSV input → schema and key validation → clean types and duplicates
          → vectorised derived columns → validated merge
          → groupby/transform summaries → plots and saved outputs
          → run metadata, row counts and reproducibility checks
```

This improves speed and makes memory use lower and more predictable. The additional checks add a small validation cost. Category conversion may not save memory when nearly every value is unique, so measure before and after. Nullable integer types can use more memory than a narrow NumPy integer but preserve missing values.

D. Predict before implementing (1 mark)

Prediction: vectorised calculation will be faster while producing the same Defect_Rate values; categorical and smaller numeric dtypes will reduce memory. Explicit checks will stop invalid inputs rather than silently generating incorrect summaries.

E. Implement and validate the redesign (2 marks)

Run baseline and redesigned pipelines on the same 100,000-row dataset and hardware. Compare wall-clock time, deep memory usage, output equality, row counts and summary equality. Run at least twice with the same input and verify identical outputs. Insert actual measurements and a comparison table after execution.

F. Justify the final design (2 marks)

Adopt the redesigned pipeline if the timing and memory targets are met and outputs remain equal. Keep the original simple approach for very small datasets where optimisation adds unnecessary complexity. If category conversion does not reduce memory for high-cardinality columns, retain the original dtype. The final decision must be based on measured results.

CONCLUSION

NumPy provides efficient array operations, broadcasting, indexing and statistical aggregation. Pandas supports practical data cleaning, indexing, merging, grouping and file handling. Visualisations communicate trends and distributions, while a reproducible pipeline makes weekly analysis repeatable and auditable. Vectorisation, appropriate dtypes and explicit validation improve efficiency and reliability, but every cleaning and modelling choice must be checked for possible bias.

APPENDIX – SUBMISSION CHECKLIST

- Attach the Python source file or notebook and the generated 1,000-record CSV.
- Include the fixed random seeds and the train/validation/test or other data partitions where used.
- Include Q1 array attributes, Fahrenheit and difference outputs, broadcasting shapes and random reproducibility check.
- Include Q2 indexing outputs, statistics, outlier positions, timing results and save/load checks.
- Include Q3 before-and-after cleaning log, data types, missing-value counts and file round-trip verification.
- Include Q4 merge indicator counts, grouped summaries, pivot table and rankings.
- Include Q5 labelled plots, four-panel figure and corrected misleading plot.
- Include Q6 runtime and memory measurements, validation failures and two-run reproducibility check.

- Include Q7 baseline-versus-redesign evidence and acknowledge external resources and AI assistance.

The supplied brief states that the report must include source code, dataset-generation procedure, raw results and plots. Students may be asked to explain indexing, cleaning bias, merge counts, visualisation choices and pipeline checks during verification.